

Year One Lecture Notes

Computer Science - Durham University

Matthew Emerson

May 2026

Contents

I	Algorithms and Data Structures	1
1	Fundamental Data Structures	1
1.1	Introduction	1
1.2	Basic Data Structures	2
2	Asymptotic Notation and Sorting	4
2.1	Asymptotic Notation	4
2.2	Sorting	5
3	Selection and Data Structures	6
3.1	Algorithms for Sorting, Searching, and Selection	6
3.2	Trees	6
3.3	Heaps	8
3.4	Lower Bounds for Sorting and Selection	10
4	Graph Theory and Algorithms	10
4.1	The Basics of Graph Theory	10
4.2	Paths, Cycles, and Connectivity	12
4.3	Trees and Isomorphism	13
4.4	Minimum Spanning Trees	16
II	Computational Thinking	18
5	Introduction to Computer Science	18
6	Logic	18
III	Computer Systems	19
7	Digital Electronics	19
7.1	Circuits	19
7.2	Introduction to Logic	20
8	Machine Architecture	21
8.1	MIPS	21
8.2	The ATmega328P Microcontroller	22
8.3	Programming in AVR Assembly	22
9	Databases	22
9.1	Database Design	22
9.2	Modelling Databases	23
9.3	Relational Calculus and Algebra	24
10	Operating Systems	25
10.1	Introduction to Operating Systems	25
10.2	Process, Memory, and Storage Management	27
IV	Mathematics for Computer Science	31

V	Programming (Black)	32
VI	Discrete Mathematics (Elective)	33
VII	References	34
11	Module Professors	34
11.1	Algorithms and Data Structures	34
11.2	Computer Systems	34
12	Cited Works	35

NUMBERS WRONG FOR COMPUTER SYSTEMS

Part I

Algorithms and Data Structures

1 Fundamental Data Structures

1.1 Introduction

1.1.1 Basic Definitions

Definition 1.1.1.1: An algorithm is a method or process that is followed to solve a problem. It must be correct, must be composed of a finite number of unambiguous steps, and must terminate.

Definition 1.1.1.2: A data structure is a particular way of storing and organising data in a computer so that it can be used efficiently.

Definition 1.1.1.3: The RAM Model of Computation is a highly simplified model of computation that we use to compute the efficiency of an algorithm. Its memory consists of an infinite array which it uses to carry out instructions sequentially. All instructions take unit time, meaning the running time is the number of instructions executed.

Definition 1.1.1.4: Pseudocode is a generic way of describing how an algorithm will function without choosing any specific programming language. It does not have any very strict rules, but follows the following generic format:

Algorithm 1 Example pseudocode

Input: Input conditions

Output: The algorithm's output

```

 $x \leftarrow 1$ 
 $y \leftarrow 2$ 
if  $y > x$  then
    return  $y$ 
else
    return  $x$ 
end if

```

Definition 1.1.1.5: A variable is used to store some value, for example, in the above code, x and y are variables. Some basic types include integer, float, char, and string.

1.1.2 Recursion and Backtracking

Definition 1.1.2.1: A recursive algorithm is an algorithm which calls itself to do a part of its work. It should have a base case (where the final result is returned), must change its state to move towards the base case, and must call itself recursively.

Definition 1.1.2.2: A backtracking algorithm tries to construct a solution to a computational problem incrementally, one small piece at a time. Whenever the algorithm needs to decide between multiple alternatives to the next component of the solution, it recursively evaluates every alternative and then chooses the best one. Definition from (Erickson, 2019, p.71).

Definition 1.1.2.3: Memoisation is the process of storing the result of a computation so that it can be subsequently retrieved without repeating the computation.

1.2 Basic Data Structures

1.2.1 Arrays

Definition 1.2.1.1: An array is a sequence of elements that is usually denoted as $A[1]$, $A[2]$, ..., $A[n]$ or as $A[1, \dots, n]$. The elements are stored in consecutive memory cells and are all of the same data type.

1.2.2 Lists

Lists can be separated into three different categories, depending on how they store data.

Definition 1.2.2.1: A singly-linked list consists of nodes, each storing an element and a pointer to the subsequent node in the list. This means that the nodes do not have to be stored consecutively.

- **Definition 1.2.2.2:** The first node in a singly-linked list is called the head.
- **Definition 1.2.2.3:** The final node in a singly-linked list is called the tail and its pointer points to null.

Definition 1.2.2.4: A doubly-linked list consists of nodes with a value and two pointers, pointing to both the previous and subsequent nodes in the list. The list also must keep a size counter to track the number of nodes in the list.

- **Definition 1.2.2.5:** At either end of the doubly-linked list is a sentinel node. These nodes either have a null next or previous pointer and do not store a value. An empty list has two sentinel nodes pointing at each other.

Definition 1.2.2.6: A circularly-linked list uses the same nodes as a singly-linked list but does not have a head or tail node. Instead, the final node points to the start of the list, creating a circle.

- **Definition 1.2.2.7:** One node in the circularly-linked list is designated as the cursor and is used as a starting point when traversing the list.

1.2.3 Stacks

Definition 1.2.3.1: A stack is a collection of objects that are inserted and removed according to the last-in-first-out (LIFO) principle. They support the following methods:

- **Compulsory Methods:**
 - **Push:** $push(element)$ inserts $element$ at the top of the stack.
 - **Pop:** $pop()$ removes and returns the top element from the stack. An error occurs if the stack is empty.
- **Optional Methods:**
 - **Size:** $size()$ returns the number of elements in the stack.
 - **Is Empty:** $isEmpty()$ returns true if the stack is empty, and false if not.
 - **Top:** $top()$ returns the top item from the stack without removing it. Throws an error if the stack is empty.

Stacks are generally implemented using an array to store the items of the stack; however this can lead to memory wastage when the stack is not full, and conversely it will throw errors if the stack is full.

1.2.4 Queues

Definition 1.2.4.1: A queue is a collection of objects that are inserted and removed according to the first-in-first-out (FIFO) principle. They support the following methods:

- **Compulsory Methods:**

- **Enqueue:** *enqueue(element)* inserts *element* at the end of the queue.
- **Dequeue:** *dequeue()* removes the element from the front of the queue. An error occurs if the queue is empty.

- **Optional Methods:**

- **Size:** *size()* returns the number of elements in the queue.
- **Is Empty:** *isEmpty()* returns true if the queue is empty, and false if not.
- **Front:** *front()* returns the element from the front of the queue without removing it. Throws an error if the queue is empty.

1.2.5 Hash Tables

Definition 1.2.5.1: A hash table consists of a bucket array and a hash function.

Definition 1.2.5.2: A bucket array for a hash table is an array A of size N , where each cell of A is thought of as a bucket storing a collection of key-value pairs.

Definition 1.2.5.3: The size N of the array is called the capacity of the hash table.

Definition 1.2.5.4: A hash function h is a function mapping each key k to an integer in the range $[0, N - 1]$, where N is the capacity of the hash table. This provides an index $h(k)$ of a location in A where the key-value pair (k, v) is stored. Hash functions are usually composed of the hash code (mapping keys to integers) and a compression function (integers to indices).

Definition 1.2.5.5: A collision occurs when two keys map to the same hash value, and consequently want to be stored in the same bucket. These can be dealt with using different methods:

- **Definition 1.2.5.6:** Separate chaining converts each bucket to a list of key-value pairs. This will result in lists of average length $\frac{N}{M}$ where N is the amount of data, and M is the capacity of the hash table.
- **Definition 1.2.5.7:** If a collision occurs at $h(k) = i$, linear probing tries to insert again at $A[(i+1)\%N]$, then if that is also full, tries at $A[(i+2)\%N]$ etc. until the key-value pair is inserted.
- **Definition 1.2.5.8:** Quadratic probing iteratively tries the buckets $A[(i + f(j)) \% N]$ for $j = 0, 1, 2, \dots$ where $f(j) = j^2$ until finding an empty bucket. This avoids clustering patterns that occur with linear probing. However, it creates secondary clustering. This strategy may not find an empty bucket in A even if one exists.
- **Definition 1.2.5.9:** Using double hashing, we choose a secondary hash function h' , and if h maps some key k to a bucket $A[i]$, with $i = h(k)$, that is already occupied, then we iteratively try the buckets $A[(i + f(j)) \% N]$ for $j = 0, 1, 2, \dots$, where $f(j) = j \cdot h'(k)$. A common choice of secondary hash function is $h'(k) = q - (k \% q)$, for some prime number $q < N$. The secondary hash function should not evaluate to 0.

Definition 1.2.5.10: Deletions from the hash table must not hinder future searches. To ensure future searches are not hindered, we use tombstones. These are markers that are left in a bucket after a deletion. If a tombstone is encountered when searching along a probe sequence to find a key, we know to continue. The use of tombstones lengthens the average probe sequence distance. Two possible remedies are:

- **Definition 1.2.5.11:** Using local reorganisation, after deleting a key, we continue to follow the probe sequence of that key and move records into the vacated bucket.

- Or, we can periodically rehash the table by reinserting all records into a new hash table. If you have a record of which keys are accessed most, these can be placed where they can be found most easily.

2 Asymptotic Notation and Sorting

2.1 Asymptotic Notation

2.1.1 Types of Complexity

Definition 2.1.1.1: The time complexity of an algorithm can be expressed as the number of basic operations executed by an algorithm when computing for a set input size.

Definition 2.1.1.2: The space complexity of an algorithm is the amount of memory required to carry out the algorithm on an input of a set size.

Definition 2.1.1.3: The worst-case time complexity of an algorithm is the largest number of basic operations possible for an input of a set size.

2.1.2 General Notation

Definition 2.1.2.1: Generally, Big-O Notation denotes that, if $f = O(g)$, then f will grow no faster than g (the upper-bound for the growth rate, or $f' \leq g'$). Formally, let f be a real or complex function, and let g be a real function such that

$$f(x) = O(g(x)) \quad \text{as } x \rightarrow \infty \quad (1)$$

meaning $f(x)$ is Big-O $g(x)$ if the absolute value of $f(x)$ is at most a positive constant multiple of the absolute value of $g(x)$ for a suitably large value of x :

$$|f(x)| \leq C \cdot |g(x)| \quad x \geq x_0, \quad x_0 \in \mathbb{R}, \quad C \in \mathbb{N} \quad (2)$$

Definition 2.1.2.2: Similarly, Big-Omega Notation denotes that, if $f = \Omega(g)$, then f will grow at least as fast as g (the lower-bound for the growth rate, or $f' \geq g'$). Formally, let f and g be functions $\mathbb{R} \rightarrow \mathbb{R}$. We say that

$$f(x) = \Omega(g(x)) \quad (3)$$

if, for a suitably large value of x , there exists a constant C such that

$$|f(x)| \geq C \cdot |g(x)| \quad x \geq x_0, \quad x_0 \in \mathbb{R}, \quad C \in \mathbb{N} \quad (4)$$

Therefore, $f(x)$ is $\Omega(g(x))$ if $g(x)$ is $O(f(x))$.

Definition 2.1.2.3: Big-Theta Notation denotes the exact growth rate of a function. This means that, for some real-valued functions f and g ,

$$f(x) = O(g(x)) \quad \text{and} \quad f(x) = \Omega(g(x)) \quad (5)$$

Definition 2.1.2.4: Little-o Notation can be used for disregarding “small-order” terms. Generally, $f(x) = o(g(x))$ if $f'(x) < g'(x)$. It is formally defined for the real-valued functions f and g such that $f(x) = o(g(x))$ when

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0 \quad (6)$$

When the limit is removed, this becomes

$$o(g) = \{f : \mathbb{N} \rightarrow \mathbb{N} \mid \forall C > 0 \exists k > 0 : C \cdot f(n) < g(n) \forall n \geq k\} \quad (7)$$

Therefore, $f(x) = o(g(x)) \implies f(x) = O(g(x))$.

Definition 2.1.2.5: Similar to Little-o Notation, Little-omega Notation is defined such that $f = \omega(g)$ if $f' > g'$, and is defined formally as

$$\omega(g) = \{f : \mathbb{N} \rightarrow \mathbb{N} \mid \forall C > 0 \exists k > 0 : f(n) > C \cdot g(n) \forall n \geq k\} \quad (8)$$

Definition 2.1.2.6: A function is designated as a sublinear function if it grows slower than any linear function. Using Little-o Notation, $f(x)$ is sublinear if $f(x) = o(x)$, meaning

$$\lim_{x \rightarrow \infty} \frac{f(x)}{x} = 0 \quad (9)$$

2.2 Sorting

2.2.1 Common Algorithms

Definition 2.2.1.1: An insertion sort works by taking the i^{th} element from a list and inserting it into an already sorted sequence of elements $[1, j - 1]$. This has a best-case runtime of $(n - 1)$ and a worst-case of $\frac{n(n-1)}{2}$, giving a Big-O of $O(n^2)$.

Definition 2.2.1.2: A selection sort takes the i^{th} smallest elements and places them in order at the start of the list until all elements have been sorted. Its time complexity can be calculated by doing the following:

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n 1 = \sum_{i=1}^{n-1} (n - i) = \left(\sum_{i=1}^{n-1} n \right) - \left(\sum_{i=1}^{n-1} i \right) = n(n-1) - \frac{n(n-1)}{2} = O(n^2) \quad (10)$$

Definition 2.2.1.3: A bubble sort takes the i^{th} largest elements and places them in order at the end of the list until all elements have been sorted. Its time complexity can be calculated by doing the following:

$$\sum_{i=1}^{n-1} (n - i) = O(n^2) \quad (11)$$

Definition 2.2.1.4: A merge sort takes a list of elements and recursively splits it into two even parts until each sublist contains one element. Once this occurs, the sublists are merged back together in the correct order to form the sorted list. It does this with a variant of an insertion sort. It has a time complexity of $O(n \log n)$.

Definition 2.2.1.5: A quick sort takes the first, middle, and final element of a list and selects one as a pivot point. The remaining elements are then placed in sublists depending on whether they are smaller/larger than the pivot. This gives a best-case time complexity of $O(n \log n)$ and a worst-case of $O(n^2)$.

2.2.2 Recursion for Sorting and Solving Recurrences

To analyse recursive algorithms, first of all note that most of them are actually hybrids between iterative and strictly recursive. For example, merge sort splits the input of size n into two halves of equal size $\frac{n}{2}$, independently recurses in each half, then merges the resulting sorted sequence.

$$T(n) \leq \begin{cases} d & \text{if } n \leq c \text{ for constants } c, d > 0 \\ 2 \cdot T(\frac{n}{2}) + a \cdot n & \text{otherwise} \end{cases} \quad (12)$$

Definition 2.2.2.1: The Master Theorem can be used to quickly solve recurrences of form

$$T(n) \leq aT\left(\frac{n}{b}\right) + f(n) \quad a \geq 1, b > 1 \quad (13)$$

The theorem has three cases:

1. $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0 \implies T(n) = \Theta(n^{\log_b a})$
2. $f(n) = \Theta(n^{\log_b a} \cdot (\log n)^k) \implies T(n) = \Theta(n^{\log_b a} \cdot (\log n)^{k+1})$
3. $f(n) = O(n^{\log_b a + \epsilon})$, $\epsilon > 0$ and $af(\frac{n}{b}) \leq cf(n)$, $c < 1 \implies T(n) = \Theta(f(n))$

3 Selection and Data Structures

3.1 Algorithms for Sorting, Searching, and Selection

3.1.1 Sorting

Definition 3.1.1.1: A bucket sort algorithm creates buckets with labels in the interval $[1, k]$ where k is the largest value in the list. Each element is then placed in its corresponding bucket before the buckets are outputted in ascending order, creating a sorted list. This gives a worst-case time complexity of $O(n + k)$. If k is small relative to n , this gives a best-case time complexity of $o(n \log n)$.

Definition 3.1.1.2: A bucket sort recursively sorts a list of items by a given digit of each element until the list is sorted (from right-most digit to left-most). This algorithm requires as many buckets as the base of the number system being used (e.g. binary requires two buckets). It has a time complexity of $O(n \log_d k)$ for a list of values in $[1, k - 1]$ in base d .

3.1.2 Searching

Definition 3.1.2.1: A binary search assumes that the list being searched is sorted. It recursively halves the search space on each pass until the desired element is found. This produces the recurrence

$$T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log n) \quad (14)$$

3.1.3 Selection

Selection algorithms are used to, for example, find the i^{th} smallest element in an unsorted list.

Definition 3.1.3.1: The quick select algorithm works similarly to the quick sort, using pivots to split the search space and reduce execution times. In the worst-case, the algorithm is $O(n^2)$, but on average executes in $O(n)$ time.

Definition 3.1.3.2: The median-of-medians algorithm is doubly recursive and works by calculating the median of the list and using it as a pivot. This gives the recurrence

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10} + 3\right) + n = O(n) \quad (15)$$

3.2 Trees

3.2.1 Binary Search Trees

Definition 3.2.1.1: A rooted binary tree is a tree where each node has a single parent node and at most two children nodes. The binary tree is rooted if there is one node with all other nodes as descendants.

Tree Properties:

- **Definition 3.2.1.2:** A tree's size is the total number of nodes in the tree.
- **Definition 3.2.1.3:** A node's depth is its distance from the root node.

Definition 3.2.1.4: A binary tree becomes a binary search tree if, for all nodes, all left-descendant nodes are smaller than it, and all right-descendant nodes are greater than it.

Theorem 3.2.1.1: Let T be a rooted binary tree of height h . Then,

- T has at most $(2^{h+1} - 1)$ nodes.
- T has at most 2^h leaves.

Proof: The maximum number of nodes is in a complete tree of height h , meaning all levels are full,

$$1 + 2 + 2^2 + 2^3 + \dots + 2^h = \sum_{n=0}^h 2^n = 2^{h+1} - 1 \quad (16)$$

The second statement follows by induction on h . Case $h = 0$ is obvious. We then consider the left and right subtrees of the root of T . Then each of them has height $\leq (h - 1)$, and (by induction hypothesis) $\leq 2^{h-1}$ leaves. Then T has at most $2 \times 2^{h-1} = 2^h$ leaves.

□

Trees can be traversed in one of three ways, these are:

- **Definition 3.2.1.5:** Preorder traversal where the root is visited, followed by the left and right subtrees.
- **Definition 3.2.1.6:** Inorder traversal where the root is visited in between the left and right subtrees.
- **Definition 3.2.1.7:** Postorder traversal where the root is visited after both the left and right subtrees.

3.2.2 Balanced Binary Search Trees

Definition 3.2.2.1: Balanced binary search trees are binary search trees which balance themselves using rotation operations to minimise height differences between subtrees.

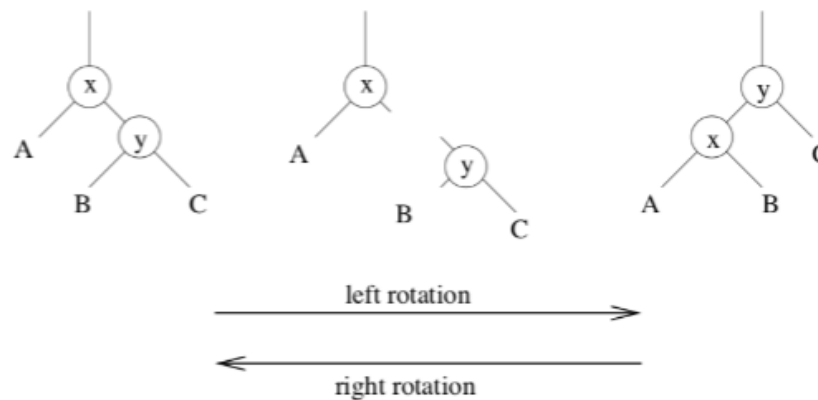


Figure 1: The basic rotation operations.

3.2.3 AVL Trees

Definition 3.2.3.1: An AVL tree is a self-balancing binary search tree with the height-balance property (for each node, the height of its children differ by at most one).

Theorem 3.2.3.1: The height of an AVL tree storing n nodes is $O(\log n)$

Proof: Let us bound $n(h)$ as the minimum number of internal nodes of an AVL tree of height h . We can clearly see that $n(1) = 1$ and $n(2) = 2$. For $n > 2$, an AVL tree of height h contains the root node, one AVL subtree of height $n(h - 1)$, and another of height $n(h - 2)$. That is,

$$n(h) = n(h - 1) + n(h - 2) \quad (17)$$

Knowing that $n(h - 1) > n(h - 2)$, we get $n(h) > 2n(h - 2)$ so, by induction,

$$n(h) > 2^i n(h - 2i) \quad (18)$$

Taking i to be $\frac{h}{2} - 1$, so that $h - 2i = 1$ or 2 , on solving we get,

$$n(h) > 2^{\frac{h}{2}-1} \quad (19)$$

Taking logarithms, $h < 2\log(n(h)) + 2$. Since $\log(n) > \log(n(h))$, the height of an AVL tree is $O(\log n)$. □

Definition 3.2.3.2: The algorithm for AVL insertion automatically rebalances the tree after inserting a new element to uphold the height-balance property. For example, for a new node n ,

- If it was inserted into the left subtree of a left child, the algorithm performs a single right rotation.
- If it was inserted into the right subtree of a right child, the algorithm performs a single left rotation.
- If it was inserted into the right subtree of a left child, the algorithm performs a left rotation followed by a right rotation.
- If it was inserted into the left subtree of a right child, the algorithm performs a right rotation followed by a left rotation.

3.3 Heaps

3.3.1 Heap Properties

Definition 3.3.1.1: A priority queue is a collection of prioritised elements that allows arbitrary element insertion and removal having the first priority (highest or lowest).

Definition 3.3.1.2: All heaps are complete binary trees. The first item in a heap is the root, the second is the left child, and the third is the right child. The next nodes always fill the next levels left-to-right.

Definition 3.3.1.3: A max heap stores values where, for every node except the root, the value is at most that of its parent. In any subtree, no values are larger than the value stored in the subtree root.

Definition 3.3.1.4: A min heap stores values where, for every node except the root, the value is at least that of its parent. In any subtree, no values are smaller than the value stored in the subtree root.

Definition 3.3.1.5: Assuming that we start counting at one for the root, the heap indices can be calculated such that, for a node at index i , the left child is at $2i$, the right child is at $2i + 1$, and the parent is at $\lfloor \frac{i}{2} \rfloor$.

3.3.2 Heap Operations

Definition 3.3.2.1: Heapify takes an incorrectly ordered heap and reorders it into an acceptable order. It has a runtime of $O(\log n)$.

Algorithm 2 Heapify

Input: A , the array to turn into a heap, v , the index of the largest value, i , the highest index.

Output: A correctly ordered heap.

```

largest  $\leftarrow v$ 
leftChild  $\leftarrow 2v$ 
rightChild  $\leftarrow 2v + 1$ 

if leftChild  $\leq i$  and  $A[\text{leftChild}] > A[\text{largest}]$  then
    largest  $\leftarrow \text{leftChild}$ 
end if
if rightChild  $\leq i$  and  $A[\text{rightChild}] > A[\text{largest}]$  then
    largest  $\leftarrow \text{rightChild}$ 
end if

if largest  $\neq v$  then
    Swap( $A[v]$ ,  $A[\text{largest}]$ )
    Heapify( $A$ , largest,  $i$ )
end if

```

Definition 3.3.2.2: Insertion adds the new element to the bottom-left of the heap before running Heapify.

Definition 3.3.2.3: Build heap takes an array of numbers and converts it into a heap by running Heapify iteratively, starting v as the size of the array and looping down to one.

Theorem 3.3.2.4: The build heap algorithm runs in $O(n)$ time.

Proof: Let $T(n)$ be the execution time for a heap of size n .

$$T(n) = \sum_{h=1}^{\log n} (\# \text{ of nodes at height } h) \cdot O(h) \quad (20)$$

Given the properties of binary trees,

$$T(n) \leq \sum_{h=1}^{\log n} \frac{n}{2^h} \cdot O(h) = O\left(n \cdot \sum_{h=1}^{\log n} \frac{h}{2^h}\right) \quad (21)$$

This is well known for $x \in \{0, 1\}$,

$$\sum_{i=0}^{\infty} i \cdot x^i = \frac{x}{(1-x)^2} \quad \sum_{h=1}^{\log n} \frac{h}{2^h} \leq \sum_{h=0}^{\infty} h \cdot \left(\frac{1}{2}\right)^h = 2 \quad (22)$$

and hence,

$$T(n) = O\left(n \cdot \sum_{h=1}^{\log n} \frac{h}{2^h}\right) = O(n) \quad (23)$$

□

3.4 Lower Bounds for Sorting and Selection

Theorem 3.4.0.1: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof: It is sufficient to determine the minimum height of a decision tree in which each permutation appears as a leaf. We can consider a decision tree of height h with l leaves corresponding to a comparison sort on n elements. Each of the $n!$ permutations of the input appears as some leaf, $l \geq n!$. Given that a binary tree of height h has at most 2^h leaves, we get that

$$n! \leq l \leq 2^h \quad (24)$$

Therefore, $n! \leq 2^h$, and after taking logs of both sides,

$$h \geq \log(n!) \equiv \Omega(n \log n) \quad (25)$$

□

Definition 3.4.0.1: An adversary algorithm is designed to intercept and limit access to the input by the original algorithm such that the time taken to make a decision is maximised. However, the adversary does not have knowledge of how the algorithm works, so must be general enough to work for any algorithm provided.

Theorem 3.4.0.2: Any comparison-based algorithm attempting to find the maximum element in an unsorted array requires at least $(n - 1)$ comparisons in the worst case.

Proof: After at most $(n - 2)$ comparisons no more than two elements can have not lost a comparison. An adversary can make either of them max and be consistent, but there is not enough information for the original algorithm to be consistent. Hence, the algorithm needs to make one more comparison, making the minimum number of comparisons $(n - 1)$.

□

Alternative Proof: Taking this into account, we can design a strategy for the adversary. The only relevant information for the algorithm is how many elements have lost at least one comparison. As soon as $(n - 1)$ elements have lost, the algorithm can decide that the only undefeated element must be the maximum.

Assuming each index i has a status (N for never lost, and L for lost), initially all indices have N . After each comparison there can be a status change $N \rightarrow L$. Therefore, the adversary must delay changes $N \rightarrow L$ as much as possible.

Given this information, each comparison will either update one status from N to L , or will update neither, providing the algorithm with at most one bit of new information. In total, the algorithm needs $(n - 1)$ bits of information to make a decision, and hence at least $(n - 1)$ comparisons are necessary.

□

4 Graph Theory and Algorithms

4.1 The Basics of Graph Theory

4.1.1 Types of Graphs

Definition 4.1.1.1: A graph G is a pair $(V(G), E(G))$ of sets where $V(G)$ is a non-empty set of vertices making up the graph and $E(G)$ is a set of unordered pairs, $\{u, v\}$, $u, v \in G$.

Definition 4.1.1.2: A directed graph is a graph where the edge set $E(G)$ instead contains ordered pairs, indicating the direction of travel along each edge.

Definition 4.1.1.3: A multi-graph allows for multiple edges to exist between vertices, meaning the edge set may contain duplicates.

Definition 4.1.1.4: A pseudo-graph allows loops, meaning pairs in the edge set can contain two instances of the same vertex.

Definition 4.1.1.5: A vertex/edge-weighted graph assigns a weight for each vertex or edge.

Definition 4.1.1.6: A complete bipartite graph is a graph such that any subgraph is a bipartite graph, meaning we can separate its vertex set into two separate sets where each edge has an endpoint in both sets.

Definition 4.1.1.7: A complete graph contains all possible edges between vertices.

Definition 4.1.1.8: The hypercube, n-dimensional cube, or n-cube is the graph with

$$V = \{(e_1, \dots, e_n) | e_i \in \{0, 1\} (i = 1, \dots, n)\} \quad (26)$$

in which two vertices are neighbours iff the corresponding rows differ in exactly one entry.

4.1.2 Other Terminology

Let G be a graph and uv be an edge in it. Then,

- **Definition 4.1.2.1:** u and v are called endpoints of the edge uv .
- **Definition 4.1.2.2:** u and v are called neighbours or adjacent vertices.
- **Definition 4.1.2.3:** uv is said to be incident to u and v .
- **Definition 4.1.2.4:** If uv is also an edge where $v \neq w$, then uv and uw are adjacent edges.

Definition 4.1.2.5: The neighbourhood of a vertex $v \in V$, denoted by $N(v)$, is the set of neighbours of v .

Definition 4.1.2.6: The degree of a vertex $v \in V$, $\deg(v)$, is the size of its neighbourhood, $|N(v)|$.

Definition 4.1.2.7: $\delta(G)$ and $\Delta(G)$ give the vertices with the smallest and largest degrees, respectively.

Definition 4.1.2.8: A vertex with degree zero is an isolated vertex.

Definition 4.1.2.9: A vertex with degree one is an end vertex or a pendant vertex.

Definition 4.1.2.10: A subgraph $G' = (V', E')$ of $G = (V, E)$ is a graph with $V' \subseteq V$ and $E' \subseteq E$.

- **Definition 4.1.2.11:** A subgraph is proper if $G' \neq G$.
- **Definition 4.1.2.12:** A subgraph is spanning if $V' = V$.
- **Definition 4.1.2.13:** An induced subgraph is a subgraph where E' contains all edges of E between vertices of V' . It is obtained by removing from G all vertices of $G \setminus G'$.

4.1.3 Basic Theorems

Theorem 4.1.3.1: Known as the Handshaking Lemma, if $G = (V, E)$ is a graph, then $\sum_{v \in V} \deg(v) = 2|E|$.

Proof: Every edge has two endpoints and contributes one to each of their degrees, so contributes two to the sum of the degrees of all vertices of V .

□

Theorem 4.1.3.2: In every undirected graph, the number of vertices of odd degree is even.

Proof: Let $G = (V, E)$ and $V_{\text{odd}} = \{v : \deg(v) \text{ is odd}\}$ and $V_{\text{even}} = \{v : \deg(v) \text{ is even}\}$. Clearly $\sum_{v \in V_{\text{even}}} \deg(v)$ is even. By the Handshaking Lemma,

$$\sum_{v \in V_{\text{odd}}} \deg(v) = 2 \cdot |E| - \sum_{v \in V_{\text{even}}} \deg(v) \quad (27)$$

it follows that $\sum_{v \in V_{\text{odd}}} \deg(v)$ is even too. Now, if we have an odd number of vertices of odd degree, then there is a contradiction, meaning there must be an even number of vertices with an odd degree.

□

Theorem 4.1.3.3: All n -dimensional cubes are bipartite.

Proof: Let K_n be an n -dimensional hypercube. Create two vertex sets, V_1 and V_2 , containing all the vertices of K_n with even and odd numbers of ones, respectively. Due to the definition of a hypercube, it is guaranteed for each edge to have an endpoint in both of the resulting vertex sets. Therefore, the n -cube must be bipartite.

□

4.2 Paths, Cycles, and Connectivity

4.2.1 Walks, Paths, and Cycles

Definition 4.2.1.1: A walk is a sequence of edges from a graph G .

Definition 4.2.1.2: A path in a graph is a subgraph which is isomorphic to the graph P_k for some integer $k \geq 1$.

Definition 4.2.1.3: A circuit is a walk where the starting node is also the ending node.

Definition 4.2.1.4: A cycle is a subgraph isomorphic to the graph C_k where $k \geq 3$.

Definition 4.2.1.5: The length of a path is the number of edges within it.

Definition 4.2.1.6: The distance between two nodes, $\text{dist}(u, v)$, is the length of the shortest path between the two vertices, and is defined as ∞ if no path exists.

Definition 4.2.1.7: The diameter of a graph is the largest possible distance between two vertices in the graph.

4.2.2 Connectivity

Definition 4.2.2.1: A graph is connected if, between every pair of vertices, there exists at least one path.

Definition 4.2.2.2: A connected component of a graph is a maximally connected subgraph.

Theorem 4.2.2.1: Every graph contains at least $|V| - |E|$ connected components.

Proof: We can prove this by induction over $m = |E|$. When $m = 0$ there are no edges, only isolated vertices, meaning there must be $|V|$ connected components.

Let $G = (V, E)$ with $|E| = m + 1$. Consider an arbitrary $e \in E$, and define $E' = E \setminus \{e\}$. By the inductive hypothesis, $G' = (V, E')$ has at least $|V| - |E'| = |V| - m$ connected components. This produces two cases: either, with e , the number of connected components does not change; or, with e , the number of connected components decreases by one. In both cases, G has at least $|V| - m - 1 = |V| - |E|$ connected components, proving the initial hypothesis.

□

Definition 4.2.2.3: A directed graph is weakly connected if the graph obtained from G by forgetting directions is connected.

Definition 4.2.2.4: A directed graph is strongly connected if any two distinct vertices are connected by paths in both directions.

Definition 4.2.2.5: A strongly connected component of a directional graph is a maximal strongly connected subgraph.

4.2.3 Special Circuits and Cycles in Graphs

Definition 4.2.3.1: A Eulerian circuit is a graph where we can start and end at the same node while traversing each edge exactly once.

Definition 4.2.3.2: A Hamiltonian cycle is a graph where we can start and end on the same node while visiting each node exactly once.

Theorem 4.2.3.1: A connected graph with at least two vertices has an Eulerian circuit iff each of its vertices has even degree.

Proof: This is proved in (Rosen, 2003, pp.634-636).

□

4.3 Trees and Isomorphism

4.3.1 Trees

Definition 4.3.1.1: A forest is an acyclic graph.

Definition 4.3.1.2: A tree is a connected forest.

Theorem 4.3.1.1: Every connected graph contains a spanning tree.

Proof: Let G be a connected graph. If G doesn't contain cycles then it is a tree and therefore is its own spanning tree. Conversely, if the graph does contain a cycle, then we can remove one edge from the graph, breaking the cycle and forming a spanning tree. The resulting graph will still be connected as cycles will contain at least one vertex of degree two, meaning removing an edge won't produce an isolated vertex.

□

Definition 4.3.1.3: A leaf is a vertex of degree one.

Theorem 4.3.1.2: Every tree on at least two vertices contains a leaf.

Proof: By contradiction, we assume that every vertex has degree zero or two. If a vertex has degree zero, then the graph is not connected, and therefore cannot be a tree. Similarly, if every vertex has degree two then we can traverse through the tree by travelling to each vertex's neighbours until we reach a node that has already been visited, proving a cycle exists, making the graph not a tree. Therefore, trees with at least two vertices must contain at least one leaf.

□

Alternative Proof: (When every vertex has a degree of at least two). Consider a longest path P and an end vertex v of P . All neighbours of v are on P as the tree must be connected, therefore if $\deg(v) \geq 2$, then there is a cycle, meaning a tree cannot exist. This holds for both endpoints of the path, proving there must be at least one vertex of degree one for the graph to be a tree.

□

Theorem 4.3.1.3: A connected graph on n vertices is a tree iff it has $(n - 1)$ edges.

Proof: For small values of n , the theorem holds, e.g., a tree with one vertex has no edges, and a tree with two vertices has one edge. Suppose each tree on $(n - 1)$ vertices has $(n - 2)$ edges. Take a tree on $n \geq 3$ vertices, T contains a leaf v . Consider the graph $T - v$, it has one vertex less and one edge less than T . $T - v$ is still acyclic and such is a tree with $n - 1$ vertices, by the inductive hypothesis it has $(n - 2)$ edges. T has one edge more, so $(n - 1)$ edges.

□

Theorem 4.3.1.4: Let T be a tree and $u, v \in V(T)$ with $u \neq v$. Then there is a unique path in T between u and v .

Proof: By contradiction, suppose there are two paths P and Q in T between u and v . Let x and y in $V(T)$ be distinct and chosen in such a way that x and y are on P and Q , but between x and y the vertices of P and Q are disjoint. Then the segments of P and Q form a cycle, contradicting that T is a tree.

□

Definition 4.3.1.4: A rooted tree is a tree in which one vertex is fixed as the root (and every other vertex is directed away from the root).

Let v be a vertex in a rooted tree T .

- **Definition 4.3.1.5:** The neighbours of v in the next level are called the children of v .
- **Definition 4.3.1.6:** The (unique) neighbour of v in the previous level is called the parent of v .
- **Definition 4.3.1.7:** If v has children, then it is an internal vertex.

Theorem 4.3.1.5: A tree with at least two vertices, n_3 of which have degree at least three, has at least $(n_3 + 2)$ leaves.

Proof: Let T be a tree on $n \geq 2$ vertices. We use induction on n . Let $l(t)$ denote the number of leaves in T , and $n_3(T)$ denote the number of vertices of degree at least three in T . If $n = 2$, $n_3 = 0$ and T has two leaves. Now suppose that every tree on fewer than n vertices has at least $(n_3 + 2)$ leaves, and consider a tree T on $n \geq 3$ vertices. Since T is a tree on at least three vertices, it has a leaf u . Then $T' = T - u$ is a tree on $(n - 1)$ vertices. By the induction hypothesis, we have

$$l(T') \geq n_3(T') + 2 \quad (28)$$

Let v be the unique neighbour of u in T . T is connected and has at least three vertices, so v has at least two neighbours in T . This results in one of three cases:

1. Suppose that v has exactly two neighbours in T . Then $n_3(T) = n_3(T')$ and $l(T) = l(T')$. Hence,

$$l(T) = l(T') \geq n_3(T') + 2 = n_3(T) + 2 \quad (29)$$

2. Suppose that v has exactly three neighbours in T . Then $n_3(T) = (n_3(T') + 1)$ and $l(T) = (l(T') + 1)$. Hence,

$$l(T) = l(T') + 1 \geq n_3(T') + 2 + 1 = n_3(T) - 1 + 2 + 1 = n_3(T) + 2 \quad (30)$$

3. Suppose that v has at least four neighbours in T . Then $n_3(T) = n_3(T')$ and $l(T) = (l(T') + 1)$. Hence,

$$l(T) = l(T') + 1 \geq n_3(T') + 2 + 1 = n_3(T) + 2 + 1 \geq n_3(T) + 2 \quad (31)$$

In each case, $l(T)$ is at least $n_3 + 2$.

□

Theorem 4.3.1.6: Every tree is a bipartite graph.

Proof: Let T be a tree and u be any one of its vertices. Place v in the set V_1 . For every vertex $u \neq v$, there is a unique path from v to u in T , consider the length of this path. If the length is odd, put u in V_2 , otherwise, place it in V_1 . V_1 and V_2 are disjoint and together make up $V(T)$ and every edge of T will have an endpoint in both V_1 and V_2 .

□

4.3.2 Isomorphism

Definition 4.3.2.1: Two graphs $G(V, E)$ and $G'(V', E')$ are isomorphic if there exists a bijective function $f : V \rightarrow V'$ such that for every $u, v \in V$, we have $uv \in E$ iff $f(u)f(v) \in E'$. Then we write $G \cong G'$.

Definition 4.3.2.2: Two rooted trees $T_1(V_1, E_1, r_1)$ and $T_2(V_2, E_2, r_2)$ are called isomorphic rooted trees if there exists an isomorphism bijection $f : V_1 \rightarrow V_2$ such that $f(r_1) = r_2$.

Definition 4.3.2.3: In a rooted tree with root r , the level, $L(i)$ is the set of vertices at distance i from the root r .

Algorithm 3 Label Vertex

Input: T , a tree, and v , a vertex from the tree.

Output: A label for vertex v .

```

if  $v$  is a leaf of  $T$  then
     $label(v) \leftarrow 10$ 
else
    for every child  $w$  of  $v$  do
         $l(w) \leftarrow LabelVertex(T, w)$ 
    end for
    Sort the labels of the children decreasingly.
     $label(v) \leftarrow 1$ 
end if
return  $label(v)$ 

```

Algorithm 4 Labelled Tree Isomorphism

Input: Two pairs of trees and roots, $(T_i, r_i) \ i \in \{1, 2\}$.

Output: YES or NO.

```

 $label(r_1) \leftarrow LabelVertex(T_1, r_1)$ 
 $label(r_2) \leftarrow LabelVertex(T_2, r_2)$ 
if  $label(r_1) = label(r_2)$  then
    return YES
else
    return NO
end if

```

Theorem 4.3.2.1: If (T_1, r_1) is isomorphic to (T_2, r_2) then Algorithm 4 returns YES.

Proof: We use induction on the number k of levels. When $k = 1$, the two trees have only their root (thus must be isomorphic), which gets label 10. Thus the algorithm returns YES. If two rooted trees are isomorphic and both have k levels then the algorithm returns YES on input. Let (T_1, r_1) and (T_2, r_2) be two isomorphic rooted trees, each with $(k + 1)$ levels.

Let f be the bijection between T_1 and T_2 . Then $r_2 = f(r_1)$ and, for every child a_1 of r_1 there exists a child a_2 of r_2 such that the subtrees rooted at a_i are isomorphic, and $a_2 = f(a_1)$. Since every such pair of subtrees are isomorphic, they both have k levels. Thus, by the induction hypothesis, Algorithm 3 returns the same label for their roots. Since all these labels are the same, Algorithm 4 returns YES.

□

4.4 Minimum Spanning Trees

4.4.1 Introduction to Minimum Spanning Trees

Definition 4.4.1.1: The minimum spanning tree problem tries to find a tree that spans all the vertices in a graph for the lowest possible cost. Minimum spanning trees have $(|V| - 1)$ edges, have no cycles, and might not be unique.

Definition 4.4.1.2: Kruskal's algorithm attempts to solve the problem by:

1. Sort the edges by weight.
2. Let $A = \emptyset$
3. Consider the edges in increasing order of weight. For each edge e , add e to A unless this would create a cycle.

It has a running time of $O(E \log V)$.

Definition 4.4.1.3: Prim's algorithm attempts to solve the problem by:

1. Let $U = \{u\}$ where u is some vertex chosen arbitrarily.
2. Let $A = \emptyset$
3. Until U contains all vertices: find the least-weight edge e that joins a vertex v in U to a vertex w not in U and add e to A and w to U .

It has a running time of $O(V \log V + E)$.

Definition 4.4.1.4: Let A be a subset of an MST. If the set $A \cup \{(u, v)\}$ is also a subset of an MST, then (u, v) is a safe edge for A .

Definition 4.4.1.5: A cut $(S, V - S)$ of $G = (V, E)$ is a partition of V .

Definition 4.4.1.6: An edge $(u, v) \in E$ crosses a cut $(S, V - S)$ if $u \in S$ and $v \in V - S$, or vice-versa.

Definition 4.4.1.7: An edge $(u, v) \in E$ is a light edge crossing a cut if its weight is smallest among all the edges crossing the cut.

Definition 4.4.1.8: A cut respects a set A of edges if no edge of A crosses the cut.

Algorithm 5 Generic MST Algorithm

Input: A graph G .

Output: An MST.

$A \leftarrow \emptyset$

while A does not form a spanning tree **do**

Find an edge (u, v) that is safe for A

$A \leftarrow A \cup \{(u, v)\}$

end while

return A

4.4.2 Implementing MST Algorithms

Algorithm 6 Implementation of Kruskal's Algorithm

Input: The set of vertices V and the set of edges E .

Output: An MST.

```

 $A \leftarrow \emptyset$ 
for Each vertex  $v \in V$  do
     $C(v) \leftarrow v$ 
end for
Sort  $E$ 
while  $E \neq \emptyset$  do
     $e \leftarrow (u, v) \in E$  with min cost
    if  $C(u) \neq C(v)$  then
         $A \leftarrow (A + e)$ 
        for Each vertex  $w \in C(u), w \in C(v)$  do
            Update  $C(w)$  with  $C(u) \cup C(v)$ 
        end for
    end if
end while
return  $A$ 

```

The sorting initially takes $O(E \log V)$ time, the union-find operations also take $O(E \log V)$ time, so the total running time is $O(E \log V)$.

Algorithm 7 Implementation of Prim's Algorithm

Input: The set of vertices V and the set of edges E .

Output: An MST.

```

 $U = \{u\}$ 
 $A \leftarrow \emptyset$ 
for Each vertex  $v \in (V - u)$  do
     $B(v)$  is the least-weight edge from  $v$  to  $U$ 
end for
while  $U \neq V$  do
    Choose  $v$  with minimum cost  $B(v)$ 
     $A \leftarrow (A + e)$ 
     $U \leftarrow (U + v)$ 
    Update  $B$ 
end while
return  $A$ 

```

To initialise the algorithm, all edges are considered. It then iterates through the while loop once per vertex extracting the minimum cost edge and performing updates in $O(\log V)$ time. This gives an overall running time of $O(V \log V + E)$.

Part II

Computational Thinking

5 Introduction to Computer Science

6 Logic

Part III

Computer Systems

7 Digital Electronics

7.1 Circuits

7.1.1 Transistors

Definition 7.1.1.1: Transistors are electronically controlled switches that turn on or off when voltage or current is applied to a control terminal.

Definition 7.1.1.2: A nMOS transistor is a sandwich of layers of conducting and insulating material. The gate is a conducting layer, on top of an insulating layer, on top of the substrate which is a semi-conductor. When its gate g is at a high voltage, current is allowed to flow from source to drain, and this is prevented when the gate is at a low voltage.

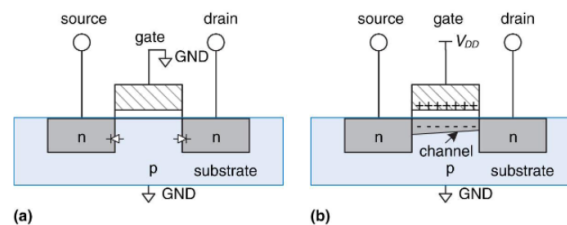


Figure 2: An nMOS transistor

The junction between p-type and n-type silicon is a diode. This means the current can only flow from p-type to n-type as they act as anodes and cathodes respectively.

Definition 7.1.1.3: A pMOS transistor is the opposite of a nMOS transistor - it allows current to flow when the gate is at low voltages, and prevents current flow at high voltages.

Definition 7.1.1.4: The static discipline states that if inputs to transistors meet valid thresholds, then the system must guarantee outputs will meet valid output thresholds and will not fall into the forbidden zone (outside of the accepted range for high/low voltages).

Definition 7.1.1.5: Moore's Law is usually quoted as “transistor density doubles every two years”, or “computer processing power doubles every eighteen months”.

7.1.2 Circuit Design

Definition 7.1.2.3: Abstraction is used when designing circuits to simplify problems by hiding unimportant details, highlighting the actual logic problem to be solved.

Definition 7.1.2.4: Another design principle used is discipline. Here, design choices are deliberately restricted to make circuits easier to model, design, and combine.

Definition 7.1.2.5: When designing complex systems, engineers use the three-y's: hierarchy, dividing a system into clear modules and submodules; modularity, using well-defined functions and interfaces for modules; and regularity, encouraging uniformity so modules can be swapped or reused.

7.1.3 Combinatorial Circuits

Definition 7.1.3.1: A combinatorial circuit is a circuit where the output depends purely on the current input.

Definition 7.1.3.2: An adder circuit takes the contents of two registers and adds them together.

- **Definition 7.1.3.3:** A half-adder circuit takes two one-bit binary inputs and adds them, returning a sum bit, and a carry bit.
- **Definition 7.1.3.4:** A full-adder circuit takes three one-bit binary inputs (one of which is a carry bit) and adds them, returning a sum bit, and a carry bit.

Definition 7.1.3.5: A decoder circuit takes an n -bit binary number and decodes it into 2^n data lines.

Definition 7.1.3.6: A multiplexor (mux) circuit uses a binary number to select one of a set of inputs to funnel to its output.

Definition 7.1.3.7: A tristate takes in two bits, an input bit and an enable bit. If the enable bit is a one, the input bit is outputted, if not then the output is left floating.

7.1.4 Sequential Circuits

Definition 7.1.4.1: A sequential circuit is a circuit which changes its output depending on its input history, alongside the current input.

Definition 7.1.4.2: A circuit which has two stable output states for a given input is known as a bistable circuit.

Definition 7.1.4.3: An sr-latch takes in two inputs, set S and reset R . If set has a pulse of one, then the output remains at one, even after the pulse ends. If reset has a pulse of one, then the output is reset back to zero. This is a bistable circuit.

Definition 7.1.4.4: A d-latch has two inputs, a data input D and a clock input CLK . The latch only changes to match the input at D when the CLK signal is high.

Definition 7.1.4.5: A d-type flip-flop works similarly to the d-latch, but only allows the stored output to change on the rising edge of the clock signal.

7.2 Introduction to Logic

7.2.1 Functionally Complete Sets

Definition 7.2.0.1: A functionally complete set is a set of boolean operators which can be used to express all possible truth tables by combining members of the set into a boolean expression. For example, $\{AND, OR, NOT\}$, $\{NOR\}$, and $\{NAND\}$ are all functionally complete sets.

8 Machine Architecture

8.1 MIPS

8.1.1 Introduction to MIPS

Definition 8.1.1.1: A MIPS (Microprocessor without Interlocked Pipeline Stages) is 32-bit RISC processor used in embedded systems and consumer electronics. It uses 32-bit words, has around 110 instructions, and 32 general purpose registers.

Definition 8.1.1.2: Amdahl's Law states that the performance improvement to be gained from using some faster mode of execution is limited by a fraction of the time the faster mode can be used.

8.1.2 The MIPS Instruction Set

The instruction set uses five addressing modes, these are:

- **Definition 8.1.2.1:** When all addresses in the instruction are registers, this is register-only addressing.
- **Definition 8.1.2.2:** When a value is used directly in an instruction, this is immediate addressing.
- **Definition 8.1.2.3:** Base addressing is when the address of a memory operand is given by a base address and a signed immediate.
- **Definition 8.1.2.4:** PC-relative addressing takes the current position in memory and jumps a given amount to a new address.
- **Definition 8.1.2.5:** Pseudo-direct addressing is used by MIPS in place of direct addressing as it does not support 32-bit addresses. As a result, it calculates it from 26 as the two least significant bits are always zeros as all words are multiples of four, and the four most significant bits can also be left as zero.

The MIPS instruction set can be split into three instruction types, these are:

- **Definition 8.1.2.6:** R-type instructions contain a six-bit opcode, three five-bit registers (two source and one destination), a five-bit shift value, and a six-bit code to identify the function.
- **Definition 8.1.2.7:** I-type instructions contain a six-bit opcode, two five-bit registers (one source, one destination), and a sixteen-bit immediate.
- **Definition 8.1.2.8:** J-type instructions contain a six-bit opcode and a twenty-six bit memory address. The compiler combines the two to form a new thirty-two bit address.

8.1.3 Memory Maps

MIPS uses 32-bit memory addresses across 2^{32} bits (or 4 GB), ranging from 0x0 to 0xFFFFFFFFC. This address space is split into the four segments:

- **Definition 8.1.3.1:** The text segment stores the machine language program and can store 256 MB.
- **Definition 8.1.3.2:** The global data segment stores global variables for program functions to access. These can be accessed using the pointer `$gp`. This segment can store 64 KB of data.
- **Definition 8.1.3.3:** The dynamic data segment stores data which is dynamically allocated through the execution of the program and can store up to 2 GB of data using a stack and a heap.
- **Definition 8.1.3.4:** The reserved segment is used by the operating system and cannot be directly accessed by the program.

8.2 The ATmega328P Microcontroller

8.2.1 Main Technical Characteristics

Definition 8.2.1.1: A microcontroller is an integrated device consisting of a microprocessor, memory, and an input/output. They are used in embedded systems and store their program in flash memory.

The microcontroller has 32 general purpose 8-bit registers that are copied onto SRAM memory. Registers 26-31 are used in addressing, and as such should be treated as three 16-bit registers, rather than six 8-bit registers.

Definition 8.2.1.2: The status register is an 8-bit register separate from the other 32. Its eight bits are used as flags.

The microcontroller's pins are separated into 3 groups, B (digital), C (analogue), and D (digital).

Definition 8.2.1.3: The direction of each pin of a port is controlled by a data direction registers. We say a bit is set when it is 1 and the corresponding pin is output, and that a bit is cleared when it is 0 and the pin is input.

8.3 Programming in AVR Assembly

8.3.1 Directives

Definition 8.3.1.1: The assembly code contains statements called directives. These do not correspond to assembly instructions, but the assembler will take them into account when compiling. In AVR assembly, a directive is preceded by a dot. Some examples include:

- *.equ* assigns a value to a label which can be used in later expressions. This is a constant. (*.equ pi, 3*).
- *.global start* and *.global btnLED* declare the functions *start* and *btnLED* as global.

9 Databases

9.1 Database Design

9.1.1 The ANSI-SPARC Architecture

The architecture, set out by ANSI (the American National Standards Institute) and SPARC(the Standards, Planning, and Requirements Committee), consists of three levels used to realise data abstraction allowing different users to access the same data in different views without affecting each other. These levels are:

- **Definition 9.1.1.1:** The external layer is the user's view of the database. It can be adapted by the user to fit their specific needs without affecting others. This layer also incorporates derived, calculated, or combined fields.
- **Definition 9.1.1.2:** The conceptual layer is the community view of the database. It describes what data is stored in the database alongside the logical structure of the entire database.
- **Definition 9.1.1.3:** The internal layer is the physical representation of the database which describes how the data is stored to achieve optimal performance and storage utilisation.

Definition 9.1.1.4: A database schema is a complete description of the database as a result of the database design process. Each database should contain multiple external schemata, a conceptual schema, and an internal schema.

Definition 9.1.1.5: A database instance is the concrete data contained within the database at any particular time.

Definition 9.1.1.6: A database is considered to have logical data independence if the external schemata remain the same if the logical structure is changed on a conceptual level.

Definition 9.1.1.7: A database is considered to have physical data independence if the conceptual schema stays the same when the internal schema is changed.

9.1.2 Designing a Database

Definition 9.1.2.1: When creating a database, the first thing to do is to create a conceptual design. This is a high-level model of the data using entity-relationship modelling, identifying the appropriate entities, attributes, relationships, and constraints. This should be independent of any physical constraints.

Definition 9.1.2.2: The next step is to create a logical design which will be used to construct the relational data model using the conceptual design as a guide to transform entities and relationships into tables, using normalisation techniques to eliminate redundancies.

Definition 9.1.2.3: The physical design should be created last, and will describe how the logical design will be implemented, defining storage, access, and security details.

9.2 Modelling Databases

9.2.1 Entity-Relationship Modelling

Definition 9.2.1.1: An entity should be explicitly represented, denoted by a labelled rectangle and split into types and instances.

Definition 9.2.1.2: A relationship is a named association between two (or more) entity types which is represented by a labelled line with a triangle indicating reading direction.

Definition 9.2.1.3: Constraints are commonly represented on an entity-relationship model by a range ($min...max$). This can also include the cardinality (one-to-one, many-to-one, etc.), and participation (optional or mandatory).

Definition 9.2.1.4: The properties of an entity type are known as its attributes. In an entity-relationship model they are placed below an entities title with the primary key underlined.

9.2.2 Relational Data Models

Definition 9.2.2.1: A relational data model is based on the concept of mathematical relations and is the most widely used data structure for designing databases. It models a database by storing the relations between data in tables.

When creating a relational data model, the design must meet the following integrity constraints:

- **Definition 9.2.2.2:** Domain constraints for attributes limit the range of values permitted.
- **Definition 9.2.2.3:** Entity integrity ensures that the primary key is not null, guaranteeing that each entity has a unique identifier and that foreign keys form a valid reference.
- **Definition 9.2.2.4:** Referential integrity ensures that all foreign keys match some primary key or is null. This ensures that all references are valid.

9.2.3 Functional Dependencies

Definition 9.2.3.1: A relation's functional dependencies describe the relationship among its attributes.

Definition 9.2.3.2: If A and B are two sets of attributes in the same relation, where A is associated with exactly one value of B , then A functionally determines B and B is functionally dependent on A . A is called the determinant of B . Functional dependencies can take one of three forms:

- **Definition 9.2.3.3:** A full functional dependency exists when B is functionally dependent on A , but not on any proper subset of A .
- **Definition 9.2.3.4:** A partial functional dependency exists when B is functionally dependent on at least one proper subset of A .
- **Definition 9.2.3.5:** A transitive functional dependency exists between A and C if $A \rightarrow B$ and $B \rightarrow C$.

Definition 9.2.3.6: Given a set of functional dependencies F , its closure F^+ are all the functional dependencies which are implied by the dependencies in F . These can be calculated using Armstrong's Axioms.

Definition 9.2.3.7: Armstrong's Axioms are:

- **Definition 9.2.3.8:** Reflexivity: If $B \subseteq A$, then $A \rightarrow B$.
- **Definition 9.2.3.9:** Augmentation: If $A \rightarrow B$, then $(A, C) \rightarrow (B, C)$.
- **Definition 9.2.3.10:** Transitivity: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

These rules are complete and sound, and can be used to derive another three rules:

- **Definition 9.2.3.11:** Decomposition: If $A \rightarrow (B, C)$, then $A \rightarrow B$ and $A \rightarrow C$.
- **Definition 9.2.3.12:** Union: If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow (B, C)$.
- **Definition 9.2.3.13:** Composition: If $A \rightarrow B$ and $C \rightarrow D$, then $(A, C) \rightarrow (B, D)$.

The closure F^+ of F can be computed by taking of F 's dependencies, repeatedly applying Armstrong's Axioms, and adding any new dependencies discovered in this way.

9.2.4 Normalisation

Definition 9.2.4.1: A table is in first normal form if it has no repeating groups (every cell has one value).

Definition 9.2.4.2: A table is in second normal form if it is in first normal form and has no partial dependencies (every non-key attribute is dependent on the whole primary key).

Definition 9.2.4.3: A table is in third normal form if it is in second normal form and has no transitive dependencies.

9.3 Relational Calculus and Algebra

9.3.1 Database Languages

Definition 9.3.1.1: Structured query language is the most common database language. It follows the ISO/ANSI standard using simple syntax with its two components, DDL and DML:

- **Definition 9.3.1.2:** Data definition language is used to define the schema for each relation, the domain for each respective attribute, and to specify the relation's integrity constraints.
- **Definition 9.3.1.3:** Data manipulation language is used to insert, update, delete, and retrieve data from the database.

Database languages can be separated into one of two types:

- **Definition 9.3.1.4:** A declarative language specifies which data needs to be retrieved in a query (relational calculus).
- **Definition 9.3.1.5:** A procedural language specifies how to retrieve the requested data for the query (relational algebra).

- **Definition 10.1.1.5:** A first-level interrupt handler which determines the source of an interrupt and initiates servicing the interrupt.
- **Definition 10.1.1.6:** The dispatcher which assigns processing resources for processes and is initiated when a current process cannot continue, or if the CPU may be used better elsewhere.
- Intra operating system communications.

10.1.2 Operating System Design

Definition 10.1.2.1: The operating system generally runs in dual-mode operation. This allows the operating system to protect itself and other system components. It differentiates which mode it is currently in using the mode bit.

- **Definition 10.1.2.2:** When the computer is in user mode it is limited as to which instructions it can execute, with some requiring switching modes to,
- **Definition 10.1.2.3:** Kernel mode (also known as system or privileged mode). In this mode, the system is given access to a set of privileged instructions which can perform tasks such as managing interrupts, performing inputs/outputs, and halting a process.

The operating system offers a set of services, some of which are:

- Providing an environment for the execution of programs and services.
- Providing a user interface, which can either be on the command line (CLI), graphical (GUI), or in Batch.
- Controlling the operation of input/output devices.
- Manipulating the file system.
- Organising communications either between processes on the same computer, or between computers over a network.
- Providing error detection, taking appropriate action to ensure correct and consistent computing.
- Allocating resources, particularly when multiple jobs are running concurrently.
- Providing protection and security by ensuring that all access to system resources are controlled and that all users are authenticated.

10.1.3 Processes and Threads

Definition 10.1.3.1: A program in execution is known as a process. A process includes code, its current activity, a stack, a data section, and a heap.

Definition 10.1.3.2: A process's information is stored in a process control block. These PCBs are then arranged together into a process table. Each PCB stores a process's unique identifier, state, memory usage, CPU scheduling information, and its corresponding CPU registers.

Definition 10.1.3.3: As a process is executed, it changes state. Here, it can take one of five states: new, when the process is being created; running, when instructions are being executed; waiting, when the process is waiting for some event to occur; ready, when the process is ready to be dispatched to the CPU; and terminated, when the process has finished executing.

Definition 10.1.3.4: A process can, as a parent process, create a number of child processes, which in turn can create other processes, forming a tree of processes. A parent process may share some of its resources with its children, and they can either execute concurrently or sequentially.

Definition 10.1.3.5: The process scheduler selects which of the currently available processes will be executed next by the CPU. It maintains the following scheduling queues:

- **Definition 10.1.3.6:** The job queue is the set of all processes in the system.
- **Definition 10.1.3.7:** The ready queue is the set of all processes residing in main memory, ready and waiting to execute.
- **Definition 10.1.3.8:** The device queues are sets of processes waiting for I/O devices.

Definition 10.1.3.9: A thread is a unit of execution which can be traced and that belongs to a process. It can either be a user or kernel thread.

The thread model of processing uses local variables within threads, with global variables being available to all threads. The model is frequently used as: it requires less time to create or terminate threads vs. processes; it may allow for execution to continue, even if part of a process is blocked; threads can share processing resources; creating threads is cheaper than creating processes or context switching; and processes can take advantage of multiprocessor architectures, increasing scalability.

Definition 10.1.3.10: Amdah's Law states that performance gains from adding additional cores with serial and parallel components follows the following equation:

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}} \quad (33)$$

where S is the serial portion and N is the number of processing cores.

For example, if an application is 75% parallel, moving from one to two cores will result in a speedup of 1.6 times. As N approaches infinity, the speedup approaches $\frac{1}{S}$.

Multithreading can be accomplished by using one of three models:

- **Definition 10.1.3.11:** The many-to-one model maps many user-level threads to a single kernel thread.
- **Definition 10.1.3.12:** The one-to-one model maps each user-level thread to a kernel thread, allowing for greater concurrency.
- **Definition 10.1.3.13:** The many-to-many model allows many user-level threads to be mapped to many kernel threads.

10.2 Process, Memory, and Storage Management

10.2.1 CPU Scheduling

Definition 10.2.1.1: The aim of multiprogramming is to maximise CPU utilisation by keeping multiple processes in memory concurrently, meaning the CPU can always be executing a ready process whilst another is waiting.

Definition 10.2.1.2: The long-term (job) scheduler selects which processes should be brought into the ready queue.

Definition 10.2.1.3: The medium-term scheduler is a part of swapping and is in-charge of handling the swapped out processes.

Definition 10.2.1.4: The short-term (CPU) scheduler selects the process to be executed next and allocates it to the CPU.

Definition 10.2.1.5: The dispatcher is the module that gives control of the CPU to the process selected by the CPU scheduler. It is in charge of context switching, switching to user mode, and jumping to the proper location in the user program to restart that program.

Definition 10.2.1.6: The dispatch latency is the time it takes for the dispatcher to stop one process and start another running.

When scheduling processes, the criteria considered are:

- CPU utilisation.
- **Definition 10.2.1.7:** Throughput, the number of processes that complete their execution per unit of time.
- **Definition 10.2.1.8:** Turnaround time, the amount of time to execute a particular process (waiting, execute, and I/O).
- **Definition 10.2.1.9:** Waiting time, the total amount of time a process has been waiting in the ready queue.
- **Definition 10.2.1.10:** Response time, the amount of time it takes from when a request was submitted until the first response.

This can be achieved using one of the following algorithms: first come, first served; shortest job first; shortest remaining time first; priority; round-robin; or multilevel/feedback queue.

10.2.2 Memory Management

Definition 10.2.2.1: Each process has a separate memory space. A pair of base and limit registers define the logical address space. The CPU must check every memory access generated in user mode to be sure it is between base and limit for that user.

Definition 10.2.2.2: The logical address space is generated by the CPU and can also be referred to as the virtual address.

Definition 10.2.2.3: The physical address space is the address seen by the memory unit. This address cannot be seen by user applications.

Definition 10.2.2.4: The memory management unit is a hardware device that maps virtual addresses to their corresponding physical ones.

Definition 10.2.2.5: Relocation registers are used to protect user processes from each other, and from changing operating-system code and data. This is known as memory protection.

Memory can be allocated into one of a set of holes using either: first-fit, putting the memory in the first hole that is big enough; best-fit, putting the memory in the smallest hole which is big enough; and worst-fit, putting the memory into the largest available hole.

Definition 10.2.2.6: External fragmentation is memory space that is able to satisfy a request but it is not contiguous.

Definition 10.2.2.7: Internal fragmentation is memory that is allocated successfully, but may be slightly larger than the requested amount of memory.

Definition 10.2.2.8: External fragmentation can be reduced by compaction, where memory contents are shuffled to place all free memory together in one large block.

Definition 10.2.2.9: Physical memory is divided into fixed-sized blocks called frame while logical memory is divided into blocks of the same size called pages.

Definition 10.2.2.10: To run a program of size n pages, we need to find n free pages before loading the program. This process is called paging.

Definition 10.2.2.11: A page table is set up to help manage the mapping of logical to physical addresses using an address translation.

Definition 10.2.2.12: The logical memory address generated by the CPU is divided into the page number, which is used as an index into the page table which contains the base address of each page in physical memory, and the page offset, which combined with the base address to define the physical memory address that is sent to the memory unit.

Definition 10.2.2.13: When a logical address is generated by the CPU, the MMU first checks if its page number is present in the TLB. If the page number is found, its frame number is immediately available and is used to access memory. If the page number is not in the TLB, a memory reference to the page table has to be created. If the TLB is already full of entries, then an existing entry is selected for replacement.

Definition 10.2.2.14: The hit ratio α , is the percentage of times that a page number is found in the associative registers.

Definition 10.2.2.15: The effective access time is calculated using

$$\text{EAT} = (\alpha \times \text{memory access time}) + 2((1 - \alpha) \times \text{memory access time}) \quad (34)$$

Definition 10.2.2.16: Memory protection is implemented by associating a protection bit with each entry. This is done by attaching a valid-invalid bit to each entry where valid indicates that the associated page is in the process's logical address space, and thus is a legal page.

Definition 10.2.2.17: The page fault rate is the percentage of page references which are invalid (represented as a decimal).

Definition 10.2.2.18: Demand paging limits the pages in memory to those which are currently required by the process during execution. This reduces the number of pages to be processed and the number of frames allocated to a process whilst increasing overall response time for a collection of processes and the number of processes executing.

Definition 10.2.2.19: Page replacement finds some page in memory that isn't really in use and swaps it out for some other required page.

Definition 10.2.2.20: Belady's anomaly states that, for some page fault algorithms, the page fault rate may increase as the number of allocated frames increases.

Definition 10.2.2.21: Thrashing occurs when a process is spending more time paging than doing actual work. If a process does not have enough pages, then the page-fault rate is very high.

Definition 10.2.2.22: Prepaging is the process of moving all the required pages into memory at once.

10.2.3 Storage Management

The performance of a disk storage device is dependent on four factors: seek time, the time to move the head to the target track; rotational delay/latency, the time waiting for the target sector to rotate under the head; access time, seek time + latency; and transfer time.

Definition 10.2.3.1: Disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

Definition 10.2.3.2: Network-attached storage is storage made available over a network rather than over a local connection.

Definition 10.2.3.3: Redundant array of inexpensive disk uses multiple disk drives to increase reliability via redundancy. It is arranged in seven levels: non-redundant striping, mirrored disks, memory-style error-correcting codes, bit-interleaved parity, block-interleaved parity, block-interleaved distributed parity, P+Q redundancy.

10.2.4 Deadlocks

Definition 10.2.4.1: A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause. A deadlock must fulfill the following conditions:

- **Definition 10.2.4.2:** Mutual exclusion, only one process at a time can use a resource.
- **Definition 10.2.4.3:** Hold and wait, a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **Definition 10.2.4.4:** No preemption, a resource can be only be released voluntarily by the process holding it, after that a process has completed its task.
- **Definition 10.2.4.5:** Circular wait, there exists a set of waiting processes such that the first process is waiting for a resource from the second process, which is waiting for a resource from the third process, and so on such that the final process is waiting on a resource from the first process.

Definition 10.2.4.6: A resource-allocation graph has a set of vertices separated into two sets, processes and resources, and a set of edges, where each of which can either be a request or assignment edge.

We can deal with deadlocks in one of the following ways:

- **Definition 10.2.4.7:** The ostrich algorithm where we simply pretend there is no problem.
- **Definition 10.2.4.8:** Deadlock prevention where processes are limited to holding one resource at a time.
- **Definition 10.2.4.9:** Deadlock avoidance where each process declares the maximum number of resources of each type that it might need.
- **Definition 10.2.4.10:** Process termination is used to recover from deadlocks by selecting a victim with the minimum cost, or rolling back to a safe state.

Definition 10.2.4.11: The system is in a safe state if there exists a sequence of all the processes in the system such that, for each process, the resources it is able to request can be satisfied from currently available resources.

Part IV

Mathematics for Computer Science

Part V

Programming (Black)

Part VI

Discrete Mathematics (Elective)

Part VII

References

11 Module Professors

The below is accurate as of the 2025/26 academic year and will not be updated in subsequent years.

11.1 Algorithms and Data Structures

Fundamental Data Structures and *Graph Theory and Algorithms* are taught by Doctor David Kutner.

- **Office Location:** Room 2011, MCS Building
- **Website:** <https://www.durham.ac.uk/staff/david-c-kutner/>
- **Preferred Contact Methods:** david.c.kutner@durham.ac.uk
- **Pronouns:** he, him

Asymptotic Notation and Sorting is taught by Professor Thomas Erlebach.

- **Office Location:** Room 2004, MCS Building
- **Website:** <https://www.durham.ac.uk/staff/thomas-erlebach/>
- **Preferred Contact Methods:** thomas.erlebach@durham.ac.uk
- **Pronouns:** he, him

Selection and Data Structures is taught by Doctor Anish Jindal.

- **Office Location:** Room 2065, MCS Building
- **Website:** <https://www.durham.ac.uk/staff/anish-jindal/>
- **Preferred Contact Methods:** anish.jindal@durham.ac.uk
- **Pronouns:** he, him

11.2 Computer Systems

Digital Electronics and *Machine Architecture* are taught by Doctor Ioannis Ivrisimtzis.

- **Office Location:** Room 2023, MCS Building
- **Preferred Contact Methods:** ioannis.ivrisimtzis@durham.ac.uk
- **Pronouns:** he, him

Databases is taught by Doctor Peter Davies-Peck.

- **Office Location:** Room 2063, MCS Building
- **Preferred Contact Methods:** peter.w.davies@durham.ac.uk
- **Pronouns:** he, him

Operating Systems is taught by Doctor Anish Jindal.

- **Office Location:** Room 2065, MCS Building
- **Preferred Contact Methods:** anish.jindal@durham.ac.uk
- **Pronouns:** he, him

12 Cited Works

Erickson, J. (2019). Algorithms. 1st ed. [online] Self-Published, p.71.

Available at: <http://jeffe.cs.illinois.edu/teaching/algorithms/> [Accessed 25 Mar. 2026].

Rosen, K.H. (2003). Instructor's Resource Guide for Discrete Mathematics and Its Applications.

Boston: McGraw-Hill, pp.634-636.